

02-AI-DECISION-TREE.md

Version : V1.0

Statut : ACTIVE

Catégorie : AI Engineering Framework

Projet : Vitala

1. Objectif

AI-DECISION-TREE.md définit les règles de décision que toute IA doit appliquer lorsqu'elle travaille sur Vitala.

Ce document transforme les situations fréquentes du développement en procédures décisionnelles claires.

Une IA ne doit pas prendre une décision importante uniquement selon son propre jugement.

Elle doit suivre l'arbre de décision approprié.

2. Principe général de décision

Avant toute décision :

Identifier le problème

↓

Chercher une règle existante

↓

Consulter la documentation

↓

Appliquer la priorité correcte

↓

Exécuter uniquement si autorisé

Si aucune règle n'existe :

L'IA DOIT signaler le manque.

Elle NE DOIT PAS créer une nouvelle règle seule.

3. Arbre principal de décision

Situation : "Je dois modifier le code"

Question 1 :

La tâche est-elle clairement définie ?

OUI :

↓

Continuer.

NON :

↓

Analyser la documentation.

↓

Si toujours ambigu :

Demander clarification.

4. Situation : "Le fichier nécessaire existe déjà"

Oui

L'IA DOIT :

- utiliser le fichier existant ;
- respecter sa structure ;
- modifier uniquement ce qui est nécessaire.

Non

L'IA doit vérifier :

1. Le fichier est-il prévu par l'architecture ?

Oui :

Créer le fichier.

Non :

Demander validation.

5. Situation : "Une nouvelle fonctionnalité est demandée"

L'IA doit vérifier :

Existe-t-elle déjà ?

↓

Oui

→ améliorer l'existant.

Non

→ vérifier l'architecture.

↓

Prévue ?

→ implémenter.

↓

Non prévue ?

→ demander décision architecture.

6. Situation : "Une solution semble meilleure que l'architecture actuelle"

L'IA ne doit pas modifier automatiquement.

Procédure :

Identifier l'amélioration

↓

Documenter le problème

↓

Proposer une solution

↓

Attendre validation

↓

Implémenter uniquement après accord

7. Situation : "Une erreur apparaît"

Procédure obligatoire :

Identifier l'erreur

↓

Lire le message complet

↓

Analyser la cause

↓

Chercher dans documentation

↓

Corriger

↓

Tester

↓

Documenter si nécessaire

L'IA ne doit jamais masquer une erreur.

8. Situation : "Un test échoue"

Décision :

Le code est incorrect

↓

Corriger le code.

Le test est incorrect

↓

Corriger le test avec justification.

La règle métier est ambiguë

↓

Demander clarification.

9. Situation : "Une dépendance externe est nécessaire"

L'IA doit vérifier :

La dépendance existe déjà ?

↓

Oui

→ utiliser celle-ci.

Non

↓

Est-elle indispensable ?

Oui

→ proposer son ajout.

Non

→ utiliser une solution interne.

L'ajout d'une nouvelle dépendance sans justification est interdit.

10. Situation : "Modification Database"

Avant toute modification :

Vérifier :

- DATABASE-DICTIONARY.md
- DATABASE-RLS-MATRIX.md
- BUILD-SUPABASE.md

Puis :

Nouvelle table ?

→ Migration obligatoire.

Nouvelle colonne ?

→ Migration obligatoire.

Modification sécurité ?

→ Vérification RLS obligatoire.

Interdit :

- modification directe en production ;
- suppression sans procédure ;
- modification historique des migrations.

11. Situation : "Modification API"

Avant création :

Vérifier :

- API-STANDARD ;
- versionnement ;
- sécurité.

Décision :

API existante ?

Oui

→ conserver le contrat.

Non

→ créer selon standards.

12. Situation : "Modification UI"

L'IA doit vérifier :

Composant existant ?

Oui

→ réutiliser.

Non

↓

Composant générique nécessaire ?

Oui

→ créer composant partagé.

Non

→ créer composant local.

13. Situation : "Plusieurs solutions possibles"

L'IA doit :

1. comparer les solutions ;
2. choisir celle qui respecte :

- 3. sécurité ;
- 4. simplicité ;
- 5. architecture ;
- 6. maintenance.

Elle doit expliquer son choix.

14. Situation : "Une information manque"

Procédure obligatoire :

```
Recherche documentation
↓
Recherche historique
↓
Recherche conventions
↓
Information trouvée ?

Oui
→ continuer.

Non
→ demander clarification.
```

L'invention est interdite.

15. Situation : "Deux documents sont contradictoires"

Appliquer :

```
AI-CONSTITUTION
↓
Architecture
```


↓

Standards techniques

↓

Documentation métier

↓

Backlog

↓

Prompt utilisateur

16. Situation : "Un utilisateur demande une modification dangereuse"

Exemples :

- désactiver sécurité ;
- supprimer protection ;
- contourner permission.

L'IA doit :

- expliquer le risque ;
- proposer une alternative sécurisée ;
- refuser l'approche dangereuse.

17. Situation : "Une tâche semble terminée"

L'IA doit vérifier :

[] Code terminé

[] Tests réalisés

[] Sécurité vérifiée

[] Documentation mise à jour

[] Architecture respectée

Seulement après :

La tâche peut être déclarée terminée.

18. Règle de décision finale

En cas de doute :

L'IA choisit toujours :

La solution la plus sûre.

La solution la plus simple.

La solution la plus maintenable.

La solution la plus compatible avec l'avenir de Vitala.

Historique

Version	Modification
V1.0	Création du système de décision IA

Fin de AI/02-AI-DECISION-TREE.md